



# Extracting Trial-Varying Latent Variables in Cognitive Models with Neural Estimators

*Ti-Fen Pan*

*UC Berkeley*

*SBI Summer school, July 2026*



# Lecture + Tutorial



01

**What use cases can the tool (LaseNet) be applied?**

Motivation, use cases, and workflow

---

02

**Hands-on tutorial**

Try out a toy example in a notebook.

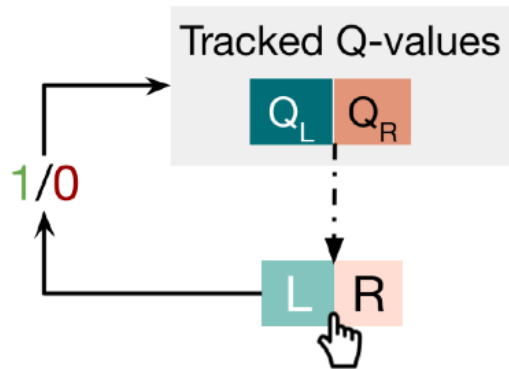


# Trial-varying Continuous Latent Variables

Use Case 1 (continuous variable) :

Reward prediction error in a Q-learning model

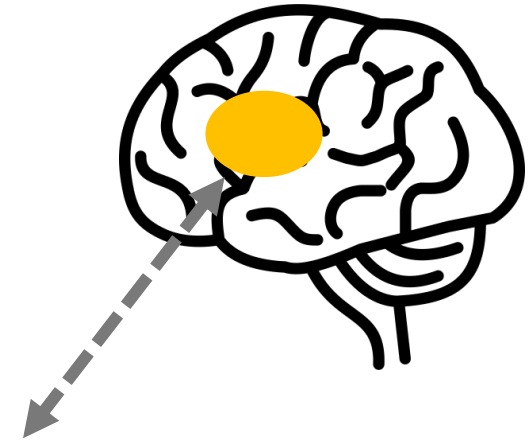
Reinforcement learning (RL)



$$\delta_t = r_t - Q_t(a_t)$$

$$Q_{t+1} = Q_t(a_t) + \alpha * \delta_t$$

$$P(L) = 1/e^{\beta(Q(R)-Q(L))}$$



$\delta_t$ : reward prediction error at trial t (**trial-varying latent variable**)

$r_t$ : reward at time t (observable variable)

$a_t$ : action at time t (observable variable)

$\alpha$ : learning rate (model parameter)

$\beta$ : choice inverse temperature (model parameter)

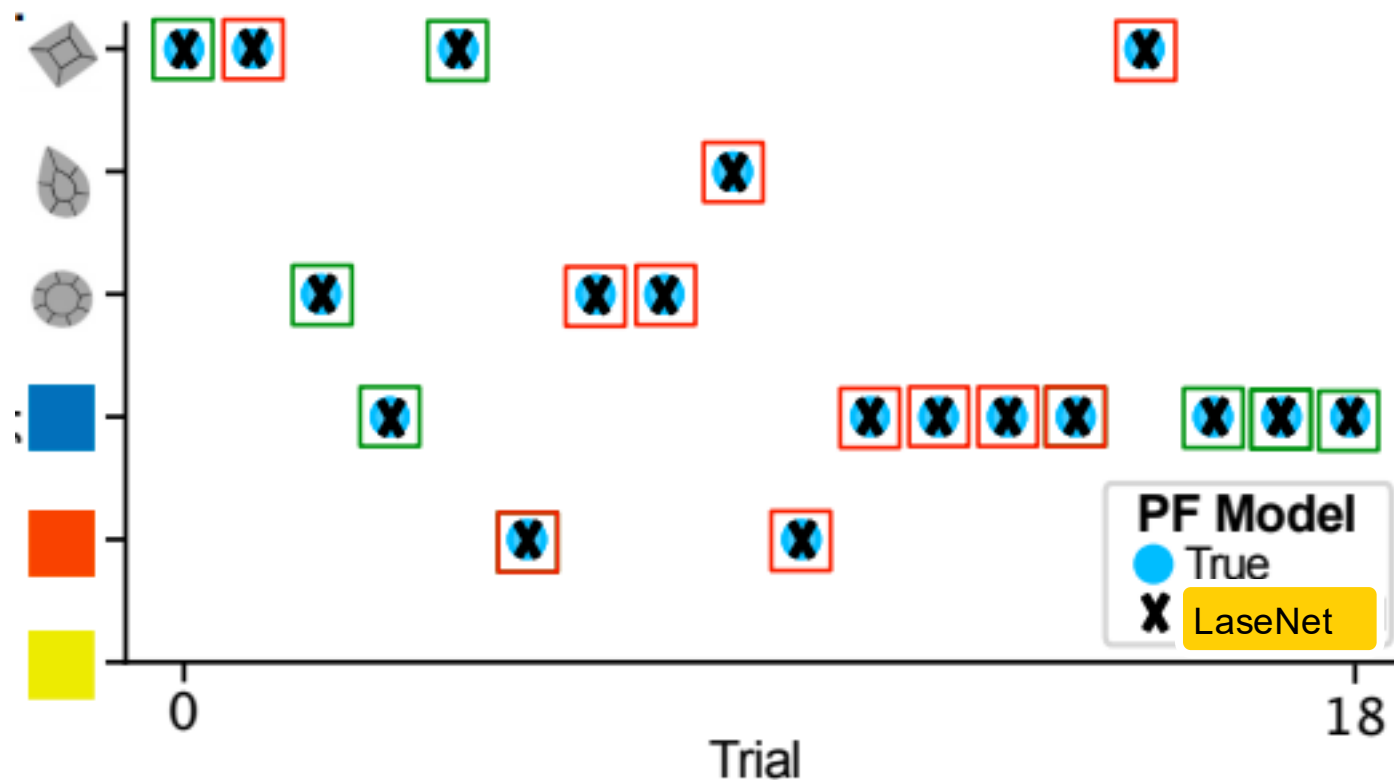
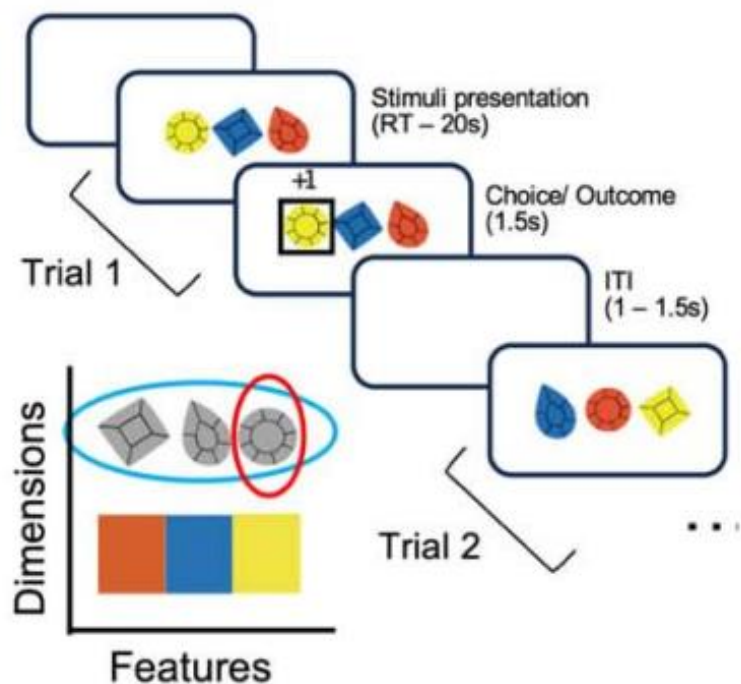
Schultz et al (1997)

Gershman et al (2024)

# Trial-varying Discrete Latent Variables

Use Case 2 (discrete variable) :

Latent stimuli feature - trial-by-trial attentional dynamics



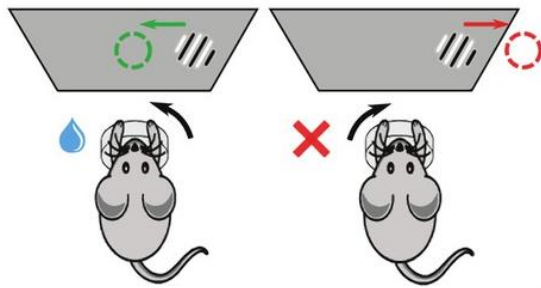
Maher et al (2025)

Learning to Attend Through Value-Based Hypothesis Testing

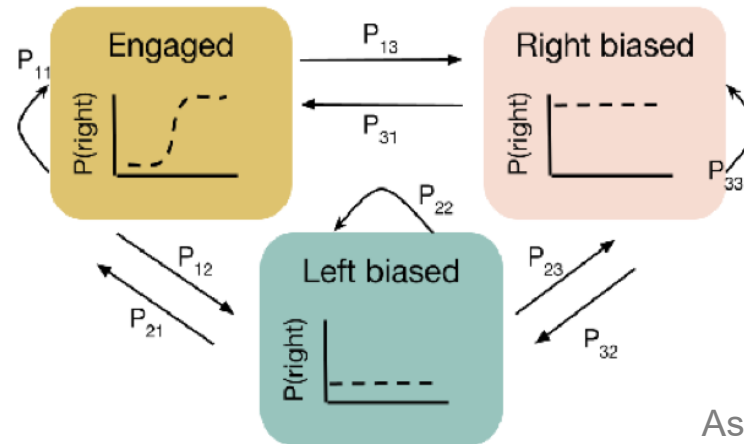
# Trial-varying Discrete Latent Variables

Use Case 3 (discrete variable)

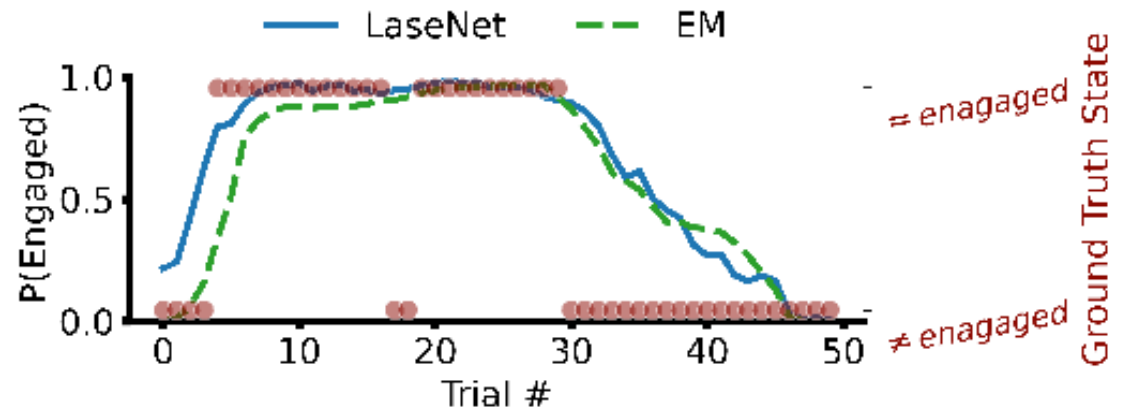
Latent states in a Bernoulli generalized linear model + Hidden Markov Model (GLM-HMM)



The International Brain Laboratory et. al (2021) eLife



Ashwood et.al. 2022  
Pan et. al 2025



# Standard Two-Steps Fitting Method

## Maximum Likelihood Estimation (MLE)

- 01 Finding the best-fitting model parameters  $\theta$ .

$$\theta_{MLE} = \underset{\theta}{\operatorname{argmax}} P(Y | \theta),$$

$Y = (y_1, y_2, \dots, y_T)$ ,  $y_t$  includes a set of stimuli, actions, and rewards (observable variables)

$$P(\theta|Y) \propto P(Y|\theta) * P(\theta)$$

Posterior  $\propto$  Likelihood \* Prior

- 02 Infer the latent variables  $z_t$  by running the computational model over participants' experienced sequences of  $Y$  with the fitted  $\theta$ .

$$z_t = f(z_{t-1}, \bar{y}_{t-1}, \theta_{MLE}),$$

$\bar{y}_{t-1}$  : the history of  $Y$  up to trial  $t-1$

Target Joint Density:  $P(z | \theta, Y) P(\theta|Y)$

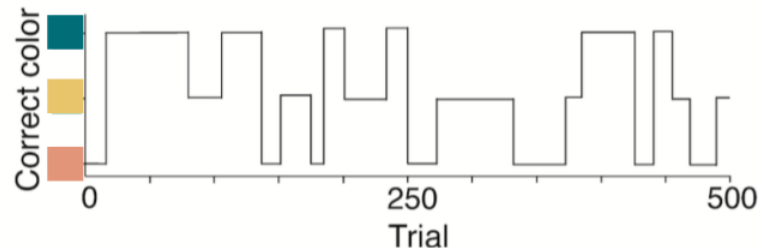
# Intractable likelihood

## Hierarchical Reinforcement Learning (HRL)

Hierarchical reversal learning task

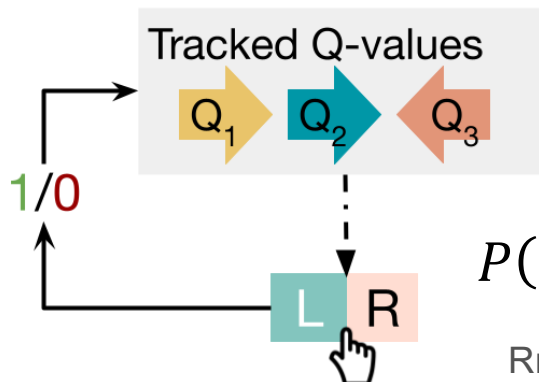


Distribution of correct rules across trials in the task



| Trial | Actions | Rewards | Chosen arrow seq |
|-------|---------|---------|------------------|
| 1     | L       | 0       | Red              |
| ...   |         |         |                  |
| ...   | ...     | ...     | ...              |

Hierarchical RL



$$P(\text{arrow}_i) \propto \exp(\beta * Q_t(\text{arrow}_i))$$

# What do researchers do if having intractable likelihood?



Approximate likelihood with...

- Complex mathematical formula (Ashwood et.al. 2021)
- Sampling method e.g., Sequential Monte Carlo/Particle Filtering. (Findling et.al, 2021)



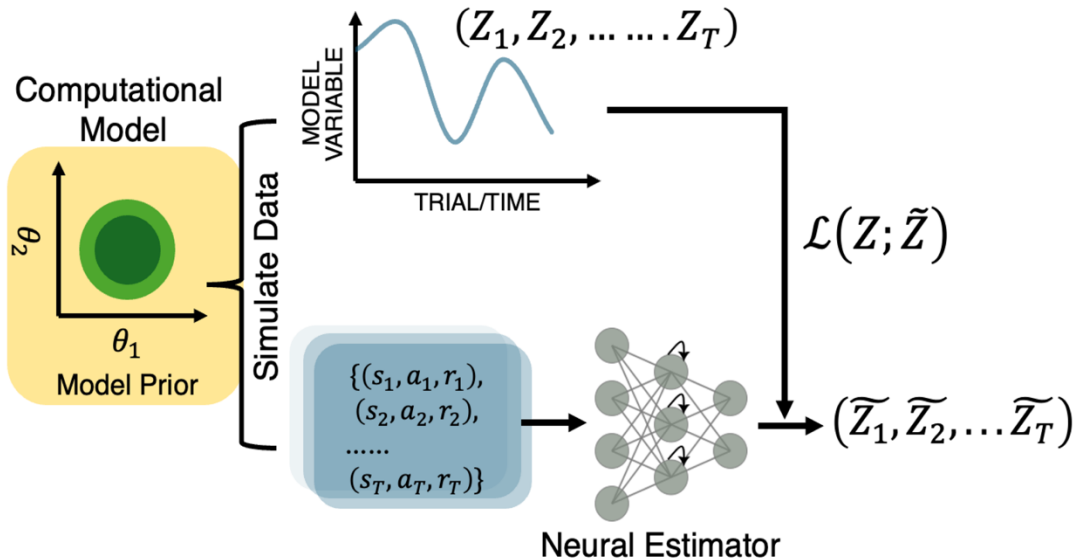
Give up and develop a model with tractable likelihood



## Proposed method

### LaseNet: Latent Sequence identification neural Networks

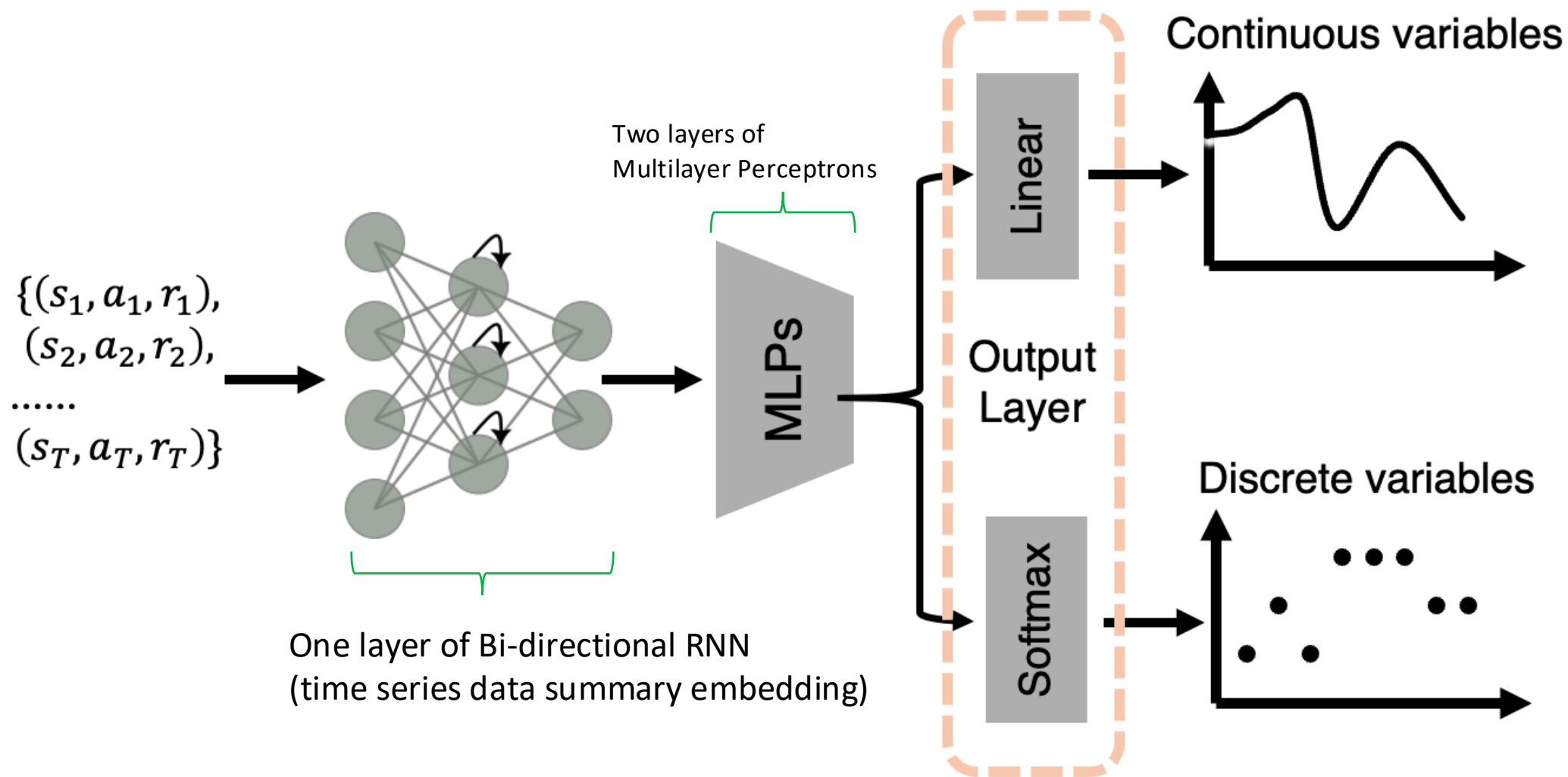
#### A) Training Phase



Target

Joint Density:  $P(z | \theta, Y) P(\theta | Y)$

## Architecture - LaseNet



## Your Turn!

1. Go to the folder “day 3 tools for SBI” on Github
2. In README, open the link under

### Recurrent networks for dynamic data (12:00)

- Instructor: Ti-Fen Pan
- Stack: Google Colab — nothing to install
- Run: open the Colab link below

LaseNet  [Open in Colab](#)

3. Go through the notebook from S0-S2 (before S3).
4. Discuss with 1 – 2 of your neighbors: what to fill in the blank at S3 (TODO)

# Acknowledgement



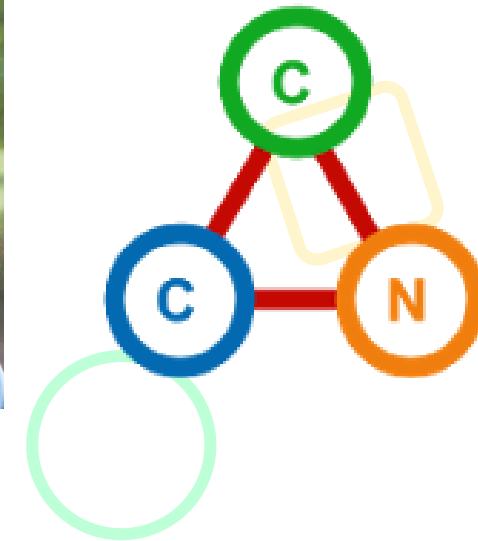
**Jing-Jing Li**



**Bill Thompson**



**Anne GE Collins**



Supported by the National Institutes of Health (NIH R21MH132974).

Behavior Research Methods (2025) 57:272  
<https://doi.org/10.3758/s13428-025-02794-0>

ORIGINAL MANUSCRIPT

**Latent variable sequence identification for cognitive models  
with neural network estimators**

Ti-Fen Pan<sup>1</sup> · Jing-Jing Li<sup>2</sup> · Bill Thompson<sup>1</sup> · Anne GE Collins<sup>1,2</sup>

Slide design credits to  Felo

Code available: <https://github.com/ti55987/lasenet>

## Evaluating Performance Against Likelihood-Dependent Methods

*Training time (mean  $\pm$  std. dev. in minutes) of LaseNet on two likelihood-intractable models that predict both continuous and discrete latent variables. Each LaseNet was trained with a learning rate of  $3e-4$  and a batch size of 256 on the L4 GPU provided by Google Colab.*

| Models       | HRL   |       |       |       | Weber-imprecision |      |      |       |
|--------------|-------|-------|-------|-------|-------------------|------|------|-------|
| # of Trials  | 720   |       |       |       | 300               |      |      |       |
| # of Samples | 600   | 1000  | 3000  | 6000  | 600               | 1000 | 3000 | 6000  |
| Mean         | 4.25  | 6.28  | 17.36 | 28.44 | 2.72              | 3.51 | 7.84 | 12.02 |
| Std. Dev     | 0.015 | 0.011 | 0.017 | 2.2   | 0.21              | 0.3  | 0.82 | 0.97  |



## What can LaseNet do now?

### Yes

- Identify latent variables within **likelihood-intractable models**.
- **Flexible** architecture that can handle both continuous and discrete latent spaces.
- **Generalizable** to a wide range of cognitive models (i.e., RL, Bayesian generative models, HMM)

### No

- **Uncertainty quantification:** different loss functions e.g., Kullback-Leibler (KL) Divergence Loss ...etc
- Infer underlying **model parameters:** using LaseNet in conjunction with other methods.